UNIT – IV

Syllabus:

Schema Refinement: Problems caused by redundancy, decompositions, problems related to decomposition, reasoning about functional dependencies, FIRST, SECOND, THIRD normal forms, BCNF, lossless join decomposition, multi-valued dependencies, FOURTH normal form, FIFTH normal form.

Schema Refinement:

Schema refinement in a Database Management System (DBMS) refers to the process of improving the database schema design to eliminate redundancy, ensure consistency, and improve query performance. This is achieved by analyzing and decomposing relations based on normalization principles and ensuring the design adheres to certain desirable properties.

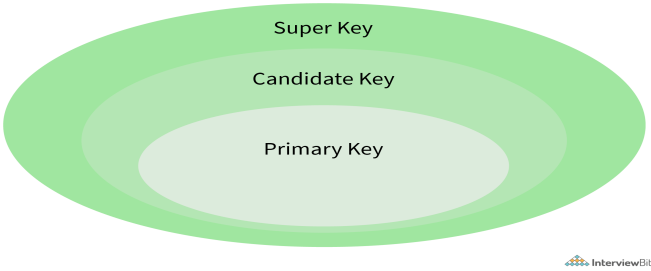
Normalization is a **scientific process** used to **reduce redundancy** in a table by decomposing it into multiple smaller tables or by adding appropriate constraints. However, it does not completely eliminate redundancy in all cases.

The **normalization process** helps to avoid **insertion, update, and deletion anomalies** and significantly reduces duplicate data.

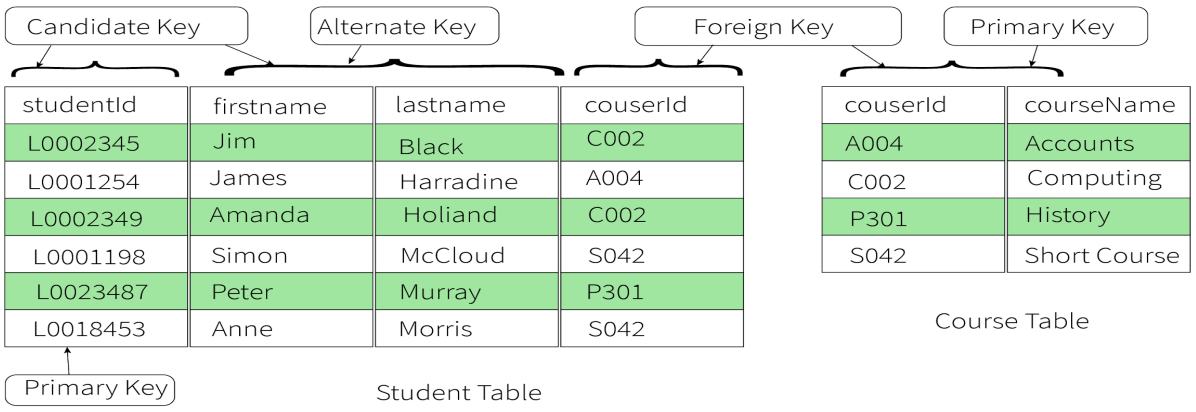
During the **design phase** of the Software Development Life Cycle (SDLC), database designers create the relations in the **logical model** of the database.

In the subsequent phase, known as **schema refinement**, they apply the **normalization process** using **normal forms** to further minimize redundancy and improve the database design.

- **Candidate Key:** The candidate key represents a set of properties that can uniquely identify a table. Each table may have multiple candidate keys. One key amongst all candidate keys can be chosen as a primary key. In the below example since studentId and firstName can be considered as a Candidate Key since they can uniquely identify every tuple.
- **Super Key:** The super key defines a set of attributes that can uniquely identify a tuple. Candidate key and primary key are subsets of the super key, in other words, the super key is their superset.



- **Primary Key:** The primary key defines a set of attributes that are used to uniquely identify every tuple. In the below example studentId and firstName are candidate keys and any one of them can be chosen as a Primary Key. In the given example studentId is chosen as the primary key for the student table.
- **Unique Key:** The unique key is very similar to the primary key except that primary keys don't allow NULL values in the column but unique keys allow them. So essentially unique keys are primary keys with NULL values.
- **Alternate Key:** All the candidate keys which are not chosen as primary keys are considered as alternate Keys. In the below example, firstname and lastname are alternate keys in the database.
- **Foreign Key:** The foreign key defines an attribute that can only take the values present in one table common to the attribute present in another table. In the below example courseId from the Student table is a foreign key to the Course table, as both, the tables contain courseId as one of their attributes.
- **Composite Key:** A composite key refers to a combination of two or more columns that can uniquely identify each tuple in a table. In the below example the studentId and firstname can be grouped to uniquely identify every tuple in the table.



Relationship Between Keys

Problems Caused by Redundancy

Redundancy occurs when the same piece of data is stored in multiple places within the database. While sometimes intentional (for performance), uncontrolled redundancy in a base schema leads to several critical problems called **Anomalies**.

Anomalies: Refers to inconsistencies or errors that occur when manipulating or querying data in a database.

These anomalies can lead to incorrect or inconsistent results and can negatively impact the overall functionality and accuracy of a database.

These include update anomalies, deletion anomalies, and insertion anomalies.

1. Insertion Anomaly

An **insertion anomaly** occurs when certain data cannot be inserted into the database without additional, unrelated data.

Example:
Consider the table Course:

StudentID	StudentName	CourseID	CourseName
201	John	C1	Math
201	John	C2	DBMS
202	Alice	C3	OS
203	Bob	C2	DBMS

- If we want to add a new course **Physics (CourseID = C4)**, but there are no students enrolled yet, we cannot insert the course because StudentID and StudentName are mandatory fields.
- If we want to add a new Student, but there are no Course enrolled yet, we cannot insert the course.

2. Update Anomaly

An update anomaly occurs when the same data is stored in multiple places, and not all instances are updated consistently, leading to data inconsistency.

Example:
Consider the table Employee:

EmpID	EmpName	DeptID	DeptName
101	Alice	D1	Sales
102	Bob	D1	Sales
103	Charlie	D2	Marketing

- If the DeptName of D1 is updated to "Global Sales," it must be changed in **all rows** where DeptID = D1.
- If we forget to update one row, the data becomes inconsistent, e.g.:

EmpID	EmpName	DeptID	DeptName
101	Alice	D1	Global Sales
102	Bob	D1	Sales
103	Charlie	D2	Marketing

The department name for D1 is inconsistent.

Solution: Decompose the table by splitting it into two tables:

1. Employee(EmpID, EmpName, DeptID)
2. Department(DeptID, DeptName)

Now, DeptName is stored only in the Department table, ensuring consistency.

3. Deletion Anomaly

A deletion anomaly occurs when deleting data inadvertently removes additional, critical information.

Example:

Consider the table Project:

ProjectID	ProjectName	EmpID	EmpName
P1	Website Build	101	Alice
P2	App Dev	102	Bob

- If we delete the P1 project, we lose all information about Alice since her data is tied only to the P1 project.
- After deletion:

ProjectID	ProjectName	EmpID	EmpName
P2	App Dev	102	Bob

There is no record of Alice anymore, even though we might want to retain employee details independent of projects.

Solution: Normalize by separating into two tables:

1. Project(ProjectID, ProjectName)
2. Employee(EmpID, EmpName, ProjectID)

This way, employee data is not lost when a project is deleted.

Decompositions

Decomposition is the process of splitting a single table (relation) with problems into multiple, smaller tables to eliminate redundancy and anomalies.

Definition: A decomposition of a relation schema R into a set of schemas (R1, R2, ..., Rn) replaces R with these new schemas such that:

- R1 ∪ R2 ∪ ... ∪ Rn = R (No attribute is lost).
- Each Ri is a subset of the attributes of R.

Example:
Let's decompose the problematic **StudentEnrollment(StuID, StuName, Major, CourseID, CourseName, Instructor, Office)** table.

Consider a poorly designed table **StudentEnrollment:**

StuID	StuName	Major	CourseID	Course Name	Instructor	Office
101	Alice	CS	CSE101	DBMS	Dr. Smith	A-101
101	Alice	CS	CSE205	OS	Dr. Jones	B-202
102	Bob	Math	MTH101	Calculus	Dr. Taylor	C-303
103	Carol	CS	CSE101	DBMS	Dr. Smith	A-101

We can split the **StudentEnrollment** table into THREE tables:

1. Student(StuID, StuName, Major)
2. Course(CourseID, CourseName, Instructor, Office)
3. And a linking table: Enrollment(StuID, CourseID)

Student Table:

StuID	StuName	Major
101	Alice	CS
102	Bob	Math
103	Carol	CS

Course Table:

CourseID	Course Name	Instructor	Office
CSE101	DBMS	Dr. Smith	A-101
CSE205	OS	Dr. Jones	B-202
MTH101	Calculus	Dr. Taylor	C-303

Enrollment Table:

StuID	CourseID
101	CSE101
101	CSE205
102	MTH101
103	CSE101

Benefits of this Decomposition:

- **Insertion Anomaly Fixed:** We can add a new student to the Student table without a course, and a new course to the Course table without a student.
- **Deletion Anomaly Fixed:** If Bob drops MTH101, we only delete the row (102, MTH101) from Enrollment. Bob's student info and the MTH101 course info remain.
- **Update Anomaly Fixed:** Dr. Smith's office is stored only once in the Course table. Changing it in that single row updates it for all associated students.

Problems Related to Decomposition

A good decomposition must satisfy two crucial properties. If it doesn't, it can introduce new problems.

Lossy Decomposition (Lossy-Join)

A decomposition is lossy (also called a lossy-join decomposition) when we cannot reconstruct the original table by joining the decomposed tables. The join operation produces extra, spurious tuples that were not in the original relation, meaning we have lost information.

Example: StudentCourse Table

Let's start with this original table:

Table: StudentCourse

StuID	StuName	CourseID	CourseName
S101	Alice	CSE101	DBMS
S101	Alice	CSE205	OS
S102	Bob	CSE101	DBMS
S103	Carol	MTH101	Calculus

Functional Dependencies:

- StuID → StuName (Student ID determines student name)
- CourseID → CourseName (Course ID determines course name)
- {StuID, CourseID} → All attributes (This is the primary key)

The Lossy Decomposition

Let's decompose this table incorrectly:

Decomposition 1: StudentInfo(StuID, StuName)

StuID	StuName
S101	Alice
S102	Bob
S103	Carol

Decomposition 2: CourseEnrollment(CourseID, CourseName, StuName)

CourseID	CourseName	StuName
CSE101	DBMS	Alice
CSE205	OS	Alice
CSE101	DBMS	Bob
MTH101	Calculus	Carol

What's Wrong with This Decomposition?

- Common attribute between the two tables: `StuName`
- Is `StuName` a superkey for either table?
    - In `StudentInfo`: `StuName` is not a superkey (we could have two students with same name)
    - In `CourseEnrollment`: `StuName` is not a superkey
- This violates the lossless join condition!

The Lossy Join in Action

Let's join these decomposed tables back together: StudentInfo ⋈ CourseEnrollment (on `StuName`)

StuID	StuName	CourseID	CourseName	
S101	Alice	CSE101	DBMS	✓ (Original tuple)
S101	Alice	CSE205	OS	✓ (Original tuple)
S102	Bob	CSE101	DBMS	✓ (Original tuple)
S103	Carol	MTH101	Calculus	✓ (Original tuple)
S101	Alice	MTH101	Calculus	🚨 SPURIOUS TUPLE!
S102	Bob	CSE205	OS	🚨 SPURIOUS TUPLE!
S103	Carol	CSE101	DBMS	🚨 SPURIOUS TUPLE!
S103	Carol	CSE205	OS	🚨 SPURIOUS TUPLE!

The Problem Revealed:

We get 4 original tuples + 4 spurious tuples = 8 tuples total!

Spurious tuples incorrectly suggest:

- Alice is enrolled in Calculus (she's not)
- Bob is enrolled in OS (he's not)
- Carol is enrolled in DBMS and OS (she's not)

This is a loss of information - we can no longer distinguish between actual enrollments and false ones.

a) The Lossless Join Decomposition

Now let's decompose it correctly using the primary key:

Decomposition 1: Student(StuID, StuName)

StuID	StuName
S101	Alice
S102	Bob
S103	Carol

Decomposition 2: Course(CourseID, CourseName)

CourseID	CourseName
CSE101	DBMS
CSE205	OS
MTH101	Calculus

Decomposition 3: Enrollment(StuID, CourseID)

StuID	CourseID
S101	CSE101
S101	CSE205
S102	CSE101
S103	MTH101

The Lossless Join Verification

Let's test if this decomposition is lossless:

Step 1: Student ⋈ Enrollment (on StuID)

- Common attribute: StuID
- Is StuID a superkey for Student? YES (it's the primary key)
- Result: Lossless join

Step 2: (Student ⋈ Enrollment) ⋈ Course (on CourseID)

- Common attribute: CourseID
- Is CourseID a superkey for Course? YES (it's the primary key)
- Result: Lossless join

Final joined result:

StuID	StuName	CourseID	CourseName
S101	Alice	CSE101	DBMS
S101	Alice	CSE205	OS
S102	Bob	CSE101	DBMS
S103	Carol	MTH101	Calculus

✓ Perfect reconstruction of the original table! No spurious tuples.

- **Problem:** When we join the decomposed tables R1 and R2 back together using a natural join (R1 ⋈ R2), we get extra tuples that were not in the original relation R. This means we have lost information.
- **Why it's bad:** The join produces spurious data, leading to incorrect query results.
- A good decomposition must be Lossless-Join.

b) Dependency Preservation

- **Problem:** The set of Functional Dependencies (FDs) that held on the original relation R might not be enforceable on the individual decomposed tables R1, R2, ...
- **Why it's bad:** To check if an update violates an FD, we might have to compute the join of multiple tables, which is computationally expensive and defeats the purpose of decomposition.
- An ideal decomposition preserves dependencies.

Reasoning about Functional Dependencies (FDs)

Functional Dependencies are the formal foundation for deciding *whether* a schema is good and *how* to decompose it properly.

Definition: A Functional Dependency X→Y holds on a relation R if, for any two tuples t1 and t2 in R, if t1[X] = t2[X] then it must be that t1[Y] = t2[Y].

In simple terms: Given a value for X, you can uniquely determine the value for Y. We say "X functionally determines Y" or "Y is functionally dependent on X."

Example: In the original StudentEnrollment table:

- StuID → StuName, Major (A student's ID determines their name and major).
- CourseID → CourseName, Instructor, Office (A course ID determines its name, instructor, and office).
- {StuID, CourseID} → All attributes (This is the key).

Importance of Functional Dependency:

1. Helps in understanding the relationship between attributes.
2. Forms the basis for normalization (removing redundancy).
3. Prevents anomalies like update, deletion, and insertion anomalies.

Types of Functional Dependencies:

1. Full Functional Dependency

Definition:
If A and B are attributes of a relation R, B is **fully functionally dependent** on A if:

- B is functionally dependent on A, and
- B is not functionally dependent on any proper subset of A.

Example:
Consider a relation Student(AdmissionNo, CourseID, Fee).

- AdmissionNo,CourseID→Fee (Full dependency)
- Fee depends on the combination of AdmissionNo and CourseID, not just one of them.

2. Partial Functional Dependency

Definition:
If A and B are attributes of a relation R, B is **partially dependent** on A if B is dependent on a **subset of A** (but not all of A).

Example:
Consider a relation Student(AdmissionNo, CourseID, CourseName).

- AdmissionNo→CourseName is a **partial dependency**, as CourseName depends only on CourseID, which is a subset of the composite key AdmissionNo,CourseID

3. Transitive Functional Dependency

Definition:
If A, B, and C are attributes of a relation R, C is **transitively dependent** on A via B if:

- A→B and
- B→C.

Example:
Consider a relation Employee(EmpID, DeptID, DeptName).

- EmpID→DeptID, and
- DeptID→DeptName.
  Thus, EmpID→DeptName (transitive dependency).

Inference Rules (Armstrong's Axioms)

These are rules to derive all FDs that are logically implied by a given set of FDs F.

1. **Reflexivity:** If $Y \subseteq X$, then $X \rightarrow Y$. (These are called trivial FDs).
   - Example: {StuID, StuName} → StuID
2. **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z.
   - Example: Given StuID → StuName, then {StuID, Major} → {StuName, Major}
3. **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
   - Example: Given CourseID → Instructor and Instructor → Office, then CourseID → Office.

Additional Rules (Derivable from Armstrong's Axioms):

1. **Union:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
2. **Decomposition:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
3. **Pseudotransitivity:** If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$.

FIRST NORMAL FORM(1NF)

A relation R is in 1NF if and only if

1. Each attribute contains atomic values (that cannot be split further)
2. Value of each attribute contains single value from domain i.e. there should be no repeating groups in a table

We consider a database of students who attend courses and score marks.

SID	NAME	CONTACT NO	COURSE	MARKS	GRADE
1	ROHAN	111-111-1111, 123-456-7890	OOPS	80	B+
1	ROHAN	111-111-1111, 123-456-7890	DBMS	95	A+
2	RAVI	222-222-3222	OOPS	75	B
2	RAVI	222-222-3222	DWDM	85	A
3	NEHA	333-333-3333	OOPS	90	A+

In the above table , **CONTACT NO** attribute contains multi values, so we can add a column to it.

SID	NAME	CONTACT NO1	CONTACT NO2	COURSE	MARKS	GRADE
1	ROHAN	111-111-1111	123-456-7890	OOPS	80	B+
1	ROHAN	111-111-1111	123-456-7890	DBMS	95	A+
2	RAVI	222-222-3222	NULL	OOPS	75	B
2	RAVI	222-222-3222	NULL	DWDM	85	A
3	NEHA	333-333-3333	NULL	OOPS	90	A+

Now, the table is in 1NF because each cell contains only atomic values.

SECOND NORMAL FORM(2NF)

A relation R is in second normal form if and only if

1. R is already in 1NF, and
2. There is no partial dependency between non-key attributes and key attributes.

Consider the following table ,

SID	NAME	CONTACT NO1	CONTACT NO2	COURSE	MARKS	GRADE
1	ROHAN	111-111-1111	123-456-7890	OOPS	80	B+
1	ROHAN	111-111-1111	123-456-7890	DBMS	95	A+
2	RAVI	222-222-3222	NULL	OOPS	75	B
2	RAVI	222-222-3222	NULL	DWDM	85	A
3	NEHA	333-333-3333	NULL	OOPS	90	A+

If we carefully observe the above relation we can find that there are four attributes that are dependent on the partial candidate key.

StudentId, Course -> Name, ContactNo1, ContactNo2, Marks, Grade
StudentId -> Name, MobileNo, HomeNo

To make this relation 2NF compliant we need to remove this partial dependency and decompose the relation.

Process: Identifying partial non key attributes which depend on “Partial Key attributes” put into a separate table. This table is called the “2NF” table. By default this table is a “Master table”. In this table only all “Non Key attributes” are “Fully Functionally dependent on total candidate key”.

The partial candidate key (StudentId) becomes the primary key of this relation.

Student Table:

SID	NAME	CONTACT NO1	CONTACT NO2
1	ROHAN	111-111-1111	123-456-7890
2	RAVI	222-222-3222	NULL
3	NEHA	333-333-3333	NULL

Marks Table:

SID	COURSE	MARKS	GRADE
1	OOPS	80	B+
1	DBMS	95	A+
2	OOPS	75	B
2	DWDM	85	A
3	OOPS	90	A+

THIRD NORMAL FORM(3NF)

A relation R is said to be in the Third Normal Form (3NF) if and only if :

1. It is in 2NF and
2. Transitive dependency does not exist between key attributes and non-key attributes through another non-key attribute.

Consider the following table ,

SID	COURSE	MARKS	GRADE
1	OOPS	80	B+
1	DBMS	95	A+
2	OOPS	75	B
2	DWDM	85	A
3	OOPS	90	A+

Let us look at the following dependencies

- **StudentId, Course** -> Marks;
- **Marks** -> Grade
- **StudentId, Course** -> Grade

Here Marks and Grade are non-key attributes while StudentId and Course are key attributes. Grade (non-key) is transitively dependent upon Student Id and Course combination (candidate key) through Marks (non-key) attribute.

We have to remove this transitive dependency to make this table 3NF compliant.

We can decompose the relation by moving the marks to grade mapping into a separate relation with Marks as the key.

Marks are also retained in the original table as a foreign key.Again there is no data loss during this decomposition as we can join these two tables to obtain the original table.

Marks Table

SID	COURSE	MARKS
1	OOPS	80
1	DBMS	95
2	OOPS	75
2	DWDM	85
3	OOPS	90

MarksGrade Table

LOMARKS	HIMARKS	GRADE
90	100	A+
85	89	A
80	84	B+
75	79	B
60	74	C+
50	59	C
35	49	D
0	34	F

BOYCE-CODD NORMAL FORM (BCNF)

Boyce-Codd Normal Form (BCNF) is an advanced version of the Third Normal Form (3NF).

A table is in BCNF if:

1. It is in 3NF.
2. For every functional dependency (FD) X→Y(the determinant) must be a superkey or candidate key.

A superkey is any set of attributes that can uniquely identify a row in a table.

When do we need BCNF?

We apply the BCNF decomposition process when a table is in **3NF** but still contains a functional dependency where a **non-key attribute** determines another **non-key attribute**, or more generally, when a determinant is not a superkey.

Steps to Check BCNF:

- Ensure the table is in 3NF.
- Check each FD to see if the determinant is a superkey.
- If any FD violates the condition, decompose the table into smaller tables to eliminate the violation.

Example Table

Consider the following table:

StudentID	CourseID	InstructorID	InstructorName
1	C101	101	Alice
2	C102	102	Bob
3	C101	101	Alice
4	C103	103	Charlie

Functional Dependencies:

1. StudentID→CourseID, InstructorID, InstructorName
2. InstructorID→InstructorName

Candidate Key: StudentID (it uniquely identifies each row).

Is the Table in BCNF?

1. Check StudentID→CourseID, InstructorID, InstructorName:
    - StudentID is a superkey. ✅
2. Check InstructorID→InstructorName:
    - InstructorID is not a superkey because it does not uniquely identify rows. ❌

Thus, the table violates BCNF due to the second FD InstructorID→InstructorName.

Decomposing into BCNF

To resolve this, split the table into two:

1. **Student-Course Table**:

StudentID	CourseID	InstructorID
1	C101	101
2	C102	102
3	C101	101
4	C103	103

Functional Dependency: StudentID→CourseID, InstructorID.

2. **Instructor Table**:

InstructorID	InstructorName
101	Alice
102	Bob
103	Charlie

Functional Dependency: InstructorID→InstructorName.

Now, both tables are in BCNF because in each table, the determinant of every functional dependency is a superkey.

Key Points:

- BCNF removes redundancy caused by functional dependencies where the determinant is not a superkey.
- Decomposition ensures data integrity and avoids anomalies like update, delete, and insert anomalies.

Multi-Valued Dependency (MVD)

Definition: In a relational database, a **multi-valued dependency** exists when, in a table, one attribute determines a set of values independently of other attributes.

OR

If a relation or table has more than one one-to-many relationship then that dependency is called multivalued dependency(MVD).

MVD can be represented as double arrows(→→).

Example-1:

Consider a table storing information about students, their enrolled courses, and their books:

COURSE	STUDENT	BOOKS
DBMS	ROHAN	DATABASE MANAGEMENT SYSTEMS
DBMS	NEHA	DATABASE SYSTEMS CONCEPTS
OOPS	ROHAN	JAVA COMPLETE REFERENCE
OOPS	RAVI	HEAD FIRST JAVA
OOPS	RAVI	FUNDAMENTALS OF OOPS
OOPS	NEHA	HEAD FIRST JAVA

Here, we observe:

- Each **Course** can be enrolled by multiple **Students**.
- Each **Course** can have multiple **Books**.

Multi-Valued Dependency:

- **Course→→Students:**
- **Course→→Books:**

Example-2:

Consider a table storing information about students, their enrolled courses, and their hobbies:

StudentID	Course	Hobby
1	Math	Painting
1	Science	Painting
1	Math	Chess
1	Science	Chess

Here, we observe:

- Each **StudentID** can have multiple **Courses**.

- Each **StudentID** can have multiple **Hobbies**.
- **Course** and **Hobby** are independent of each other for the same **StudentID**.

Multi-Valued Dependency:

- StudentID→→Course: A student's courses are independent of their hobbies.
- StudentID→→Hobby: A student's hobbies are independent of their courses.

Key Points:

- **MVDs** occur when one attribute determines multiple independent attributes.
- To handle MVDs, we normalize the table into **Fourth Normal Form (4NF)**, where no MVDs are allowed unless they are trivial (i.e., one of the attributes involved is a candidate key).

FOURTH NORMAL FORM(4NF):

If a table is in 4NF that should satisfy the following two conditions:

1. It should be in the Boyce-Codd Normal Form,and,
2. The table should not have any Multi-Valued Dependency

Example-1:

Consider a table storing information about students, their enrolled courses, and their books:

COURSE	STUDENT	BOOKS
DBMS	ROHAN	DATABASE MANAGEMENT SYSTEMS
DBMS	NEHA	DATABASE SYSTEMS CONCEPTS
OOPS	ROHAN	JAVA COMPLETE REFERENCE
OOPS	RAVI	HEAD FIRST JAVA
OOPS	RAVI	FUNDAMENTALS OF OOPS
OOPS	NEHA	HEAD FIRST JAVA

Here, we observe:

- Each **Course** can be enrolled by multiple **Students**.
- Each **Course** can have multiple **Books**.

Multi-Valued Dependency:

- **Course→→Students:**
- **Course→→Books:**

Converting to 4NF

To eliminate non-trivial MVDs, decompose the table into two tables:

COURSE_BOOKS Table

COURSE	BOOKS
DBMS	DATABASE MANAGEMENT SYSTEMS
DBMS	DATABASE SYSTEMS CONCEPTS
OOPS	JAVA COMPLETE REFERENCE
OOPS	HEAD FIRST JAVA
OOPS	FUNDAMENTALS OF OOPS
OOPS	HEAD FIRST JAVA

COURSE_STUDENT Table

COURSE	STUDENT
DBMS	ROHAN
DBMS	NEHA
OOPS	ROHAN
OOPS	RAVI
OOPS	RAVI
OOPS	NEHA

Example-2

Consider a table where we store information about students, their enrolled courses, and their hobbies:

StudentID	Course	Hobby
1	Math	Painting
1	Science	Painting
1	Math	Chess
1	Science	Chess

Observations:

1. A **StudentID** can have multiple **Courses** (e.g., 1 → Math, Science).
2. A **StudentID** can have multiple **Hobbies** (e.g., 1 → Painting, Chess).
3. **Course** and **Hobby** are independent of each other.

Here, the multi-valued dependencies are:

- StudentID→→Course
- StudentID→→Hobby

This table is not in 4NF because it has non-trivial MVDs.

Problems with This Table

- **Redundancy**: Each combination of courses and hobbies results in duplicate rows.
- **Anomalies**:
    - If a student adds a new hobby, multiple rows need to be added for each course.
    - Deleting one course might unintentionally remove information about a student's hobby.

Converting to 4NF

To eliminate non-trivial MVDs, decompose the table into two tables:

1.Student-Course Table:

StudentID	Course
1	Math
1	Science

This resolves the MVD StudentID→→Course.

2. Student-Hobby Table:

StudentID	Hobby
1	Painting
1	Chess

This resolves the MVD StudentID→→Hobby.

Benefits of 4NF

- Eliminates redundancy caused by multi-valued dependencies.
- Prevents anomalies in data insertion, deletion, and updating.
- Ensures data integrity.

Now, both tables are in 4NF, as they have no non-trivial MVDs. Each table represents a single, independent relationship.

Lossless Join Decomposition or Join Dependencies

Lossless join decomposition is a property of database decomposition that ensures no data is lost when the decomposed relations are joined back together. The decomposition is considered **lossless** if we can reconstruct the original relation without introducing spurious tuples (incorrect or extra data) by performing a natural join on the decomposed relations.

Key Conditions for Lossless Join Decomposition

To achieve a lossless join, the decomposition of a relation R into sub-relations R1 and R2 must satisfy:

1. The intersection of R1 and R2 must not be empty ($R1 \cap R2 \neq \varnothing$).
2. The common attributes ($R1 \cap R2$) must form a superkey in at least one of the relations R1 or R2.

Example

Step 1: Original Relation

Let R(A,B,C) be a relation with the following functional dependencies (FDs):

- $A \rightarrow B$
- $B \rightarrow C$

Step 2: Decompose R

We decompose R into two relations R1 and R2:

- R1(A,B)
- R2(B,C)

Step 3: Check for Lossless Join

1. Find the common attribute(s):
   $R1 \cap R2 = \{B\}$
2. Check if B is a superkey in either R1 or R2:
   - In R1(A,B), $A \rightarrow B$, so A is a superkey, but B alone is not.
   - In R2(B,C), B is a determinant ($B \rightarrow C$), making B a superkey in R2.

Since B is a superkey in R2, the decomposition is **lossless**.

Step 4: Verification :

If we join R1 and R2 using a natural join, we can reconstruct R(A,B,C) without losing any information or introducing spurious tuples.

Conclusion

Lossless join decomposition ensures that splitting a relation does not compromise the integrity or completeness of the data when the decomposed relations are recombined.

Fifth Normal Form (5NF)

The **Fifth Normal Form (5NF)**, also called **Project-Join Normal Form (PJNF)**, ensures that a database schema is free from redundancy by addressing **multi-valued dependencies** and ensuring that a table cannot be further decomposed without losing information.

A relation is in 5NF if:

- It is in **4NF**.
- Does not contain any **join dependencies(cyclic relationships)** In simpler terms, It cannot be decomposed into two or more relations without loss of information, even when dealing with multi-valued dependencies.

Example of 5NF

Initial Table:

Consider the following table that tracks employees, the projects they work on, and the skills they use:

Employee	Project	Skill
Alice	Project1	Java
Alice	Project1	SQL
Alice	Project2	Java
Bob	Project2	Python
Bob	Project3	SQL

The Problem (Redundancy): At first glance, this looks fine. But there is a hidden redundancy caused by a cyclic relationship. The fact that "Alice works on Project 1 using Java" is actually a combination of three independent facts:

1. Alice is assigned to Project1.
2. Project1 requires Java.
3. Alice knows Java.

If Alice learns a new skill (e.g., Python) and Project2 needs Python, we have to insert a new row (Alice, Project2, Python) into this big table. If we forget, our data is incomplete. 5NF solves this by separating these independent facts.

Problems in the Initial Table

1. **Redundancy**: The same employee-skill pair appears multiple times for different projects.
2. **Update Anomalies**: Adding a new skill for an employee would require multiple updates for every project the employee is working on.
3. **Deletion Anomalies**: If Alice stops working on Project1, we might accidentally lose her skill information.

Decomposition into 5NF

To eliminate redundancy, we decompose the table into three smaller tables that handle the independent relationships:

Table A : Employee-Project Table((Who is working on what?)):
This table tracks assignments only.

Employee	Project
Alice	Project1
Alice	Project2
Bob	Project2
Bob	Project3

Table B : Project_Skill (What skills does the project require?):

This table tracks project requirements.

Project	Skill
Project1	Java
Project1	SQL
Project2	Java
Project2	Python
Project3	SQL

Table C : Employee-Skill Table (What skills does the employee possess?):
This table tracks employee competency.

Employee	Skill
Alice	Java
Alice	SQL
Bob	Python
Bob	SQL

Reconstruction Logic: Combine the **Employee-Project,Project_Skill** and **Employee-Skill** tables to reconstruct the original data.

Why is this 5NF? (The Logic)

The original table violates 5NF because it has a **Join Dependency**. This means the data is only accurate if we look at all three relationships simultaneously.

The "Triangle" Rule: In this scenario, a row exists in the main table **IF AND ONLY IF**:

1. The Employee works on the Project (Table A).
2. **AND** The Project needs the Skill (Table B).
3. **AND** The Employee has the Skill (Table C).

Verification (Re-joining): If you join Table A, Table B, and Table C together using SQL, you will get **exactly** the original table back.

1. **Project 2 Example:**
    - **Table A says:** Alice is on Project2.
    - **Table B says:** Project2 needs Java AND Python.
    - *So far, potential rows:* `(Alice, P2, Java)` and `(Alice, P2, Python)`.
    - **Table C says:** Alice knows Java (but implies she doesn't know Python).
    - **Result:** The system keeps `(Alice, P2, Java)` but **discards** `(Alice, P2, Python)`.

Conclusion:

- The **Original Table** was **NOT in 5NF** because it combined these three distinct relationships into one, leading to potential inconsistencies.
- The **Three Decomposed Tables** are **IN 5NF** because they store each relationship separately. We can now add a skill to `Employee_Skill` without touching the project assignments, and the database logic holds true.

Benefits of 5NF

1. **No Redundancy**: Each relationship is stored exactly once.
2. **Avoids Update and Deletion Anomalies**: Changes in employee skills or project assignments are made in one place without risking data inconsistency.
3. **Preservation of Dependencies**: Data integrity is maintained by ensuring all dependencies are captured.

5NF ensures that a relation is decomposed into smaller relations without redundancy and without losing information, even when dealing with multi-valued dependencies. This is particularly useful in complex databases where attributes can have independent relationships.

With Join Dependency :

You split the table into **Two Tables**: `(Employee, Project)` and `(Employee, Skill)`.

Table 1 : Employee-Project Table((Who is working on what?)):
This table tracks assignments only.

Employee	Project
Alice	Project1
Alice	Project2
Bob	Project2
Bob	Project3

Table 2 : Employee-Skill Table (What skills does the employee possess?):
This table tracks employee competency.

Employee	Skill
Alice	Java
Alice	SQL
Bob	Python
Bob	SQL

Why this fails as a 5NF explanation

5NF deals with **Join Dependency** (cyclic relationships). For a table to require 5NF, there must be a **constraint** that relates all three entities (Employee, Project, and Skill) in a way that *two* tables cannot capture.

The "Spurious Tuple" Proof:

Let's look at your proposed solution (Two Tables) and try to join them back together.

- **Table 1 (Emp-Proj):** Alice works on Project 1, Alice works on Project 2.
- **Table 2 (Emp-Skill):** Alice knows Java, Alice knows SQL.

If you join these two tables, you get every combination:

1. Alice, Project 1, Java (Matches original)
2. Alice, Project 1, SQL (Matches original)
3. Alice, Project 2, Java (Matches original)
4. **Alice, Project 2, SQL ← FAKE ROW (Spurious Tuple)**

Look at your original table:

Alice | Project2 | Java exists.

Alice | Project2 | SQL does not exist.

By splitting it into only two tables, you lost the specific rule that "Project 2 does not require SQL" or that "Alice didn't use SQL for Project 2." You created false data.